Reviewer Pack — Minimal Rank of Quantum Logarithmic Forms vs Sum-of-Squares …

Entry 42c80e169aa6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 23:57:54 UTC

Minimal Rank of Quantum Logarithmic Forms vs Sum-of-Squares Degree for Max-CUT

Entry ID: `42c80e169aa6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 23:57:54 UTC

1. Conjecture

Field A (mathematical branch): Quantum Logarithmic Forms **Field B** (complexity object): Sum-of-squares Degree for Max-CUT

Statement:

> ['For every quantum logarithmic form with coefficients in a finite field F_q, its minimal rank over the algebraic closure of F_q is upper-bounded by the sum-of-squares degree of any 0.879-approximator for max-CUT.', 'This bound holds true when the quantum logarithmic form is evaluated at the zeros of the characteristic polynomial of a random matrix in GL_n(F_q).', 'For all instances with n ≤ 40, if there exists an 0.879-approximator with sum-of-squares degree less than the minimal rank of the corresponding quantum logarithmic form, then such an instance is not hard for max-CUT.']

Rationale (proposer's reasoning):

> ['Quantum logarithmic forms offer a bridge between quantum information theory and algebraic geometry, potentially revealing new insights into the computational hardness of problems like max-CUT.', 'The minimal rank of a quantum logarithmic form can be computed efficiently using the Gram-Schmidt process, providing a concrete mathematical object to study.', 'This conjecture suggests that properties of quantum logarithmic forms could be used to derive lower bounds on the sum-of-squares degree for Max-CUT problems.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b6cd559c9c10111b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> A quantum logarithmic form's minimal rank over F_q's algebraic closure is less than or equal to the sum-of-squares degree of a 0.879-approximator for max-CUT when evaluated at zeros of its characteristic polynomial, for all n ≤ 40 instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quantum logarithmic forms AND sum-of-squares degree FOR Max-CUT - upper bound ON minimal rank OF quantum logarithmic forms BY sum-of-squares degree FOR Max-CUT - characteristic polynomial OF random matrix IN GL_n(F_q) AND evaluation ON quantum logarithmic form

Top relevant hits considered: - [http://arxiv.org/abs/2512.10504v2] Tianyan: Cloud services with quantum advantage - [http://arxiv.org/abs/2601.14282v1] Quantum scientists for disarmament: a manifesto - [http://arxiv.org/abs/2307.02593v2] Superpositions of thermalisations in relativistic quantum field theory - [http://arxiv.org/abs/2404.18142v1] Revisiting Majumdar-Ghosh spin chain model and Max-cut problem using variational quantum algorithms - [http://arxiv.org/abs/1606.04387v2] Low-Rank Sum-of-Squares Representations on Varieties of Minimal Degree - [http://arxiv.org/abs/2105.12016v4] Waring ranks of sextic binary forms via geometric invariant theory - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/1410.0939v3] The characteristic polynomial of a random unitary matrix and Gaussian multiplicative chaos - The L^2-phase

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q = 2  # Example finite field F_q, can be changed to other primes if needed
    n = random.randint(5, 40)
    instances_tested = 100  # Ensure at least 30 instances per seed for statistical signal

    def generate_max_cut_instance(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def characteristic_polynomial(matrix):
```

```
        det = 0
        sign = 1
        for p in itertools.permutations(range(n)):
            term = sign * matrix[p[0]][p[0]]
            for i in range(1, n):
                term *= matrix[p[i]][p[(i + p[0]) % n]]
            det += term
            sign *= -1
        return det

    def gram_schmidt(matrix):
        q = len(matrix)
        rank = 0
        for i in range(q):
            if any(matrix[j][i] != 0 for j in range(i)):
                rank += 1
                matrix[i][:] = [matrix[i][j] / matrix[i][i] for j in range(q)]
                for j in range(i + 1, q):
                    matrix[j][:] = [matrix[j][k] - matrix[j][i] * matrix[i][k] for k in range(q)]
        return rank

    def sum_of_squares_degree(n):
        # Placeholder function, replace with actual implementation
        return n

    total_rank = 0
    total_sum_of_squares = 0

    for _ in range(instances_tested):
        max_cut_instance = generate_max_cut_instance(n)
        matrix = [[random.randint(0, q - 1) for _ in range(n)] for _ in range(n)]
        det = characteristic_polynomial(matrix)
        rank = gram_schmidt(matrix)
        sum_of_squares = sum_of_squares_degree(n)

        total_rank += rank
        total_sum_of_squares += sum_of_squares

    avg_rank = total_rank / instances_tested
    avg_sum_of_squares = total_sum_of_squares / instances_tested

    conjecture_holds = avg_rank >= avg_sum_of_squares
    counterexample = "" if conjecture_holds else f"avg_rank={avg_rank}, avg_sum_of_squares={avg_su

    return {
        "metric_name": "Rank vs Sum-of-Squares Degree",
        "metric_value": avg_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 8
    results = []
```

```python
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"avg_rank < avg_sum_of_squares\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete within the allotted time limit. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11980
2	preregistration	ollama_remote	glm4:latest	0	0	5434
3	novelty	ollama_remote	glm4:latest	0	0	4851
4	novelty	ollama_remote	glm4:latest	0	0	6937
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18371
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10707
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10798
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10519
9	judge	ollama_remote	glm4:latest	0	0	8286

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 87883 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/42c80e169aa6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/42c80e169aa6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/42c80e169aa6.tar.gz` (if generated)